ML-DSA-OSH: An Efficient, Open-Source Hardware Implementation of ML-DSA (Full Version)

Quinten Norga
COSIC
KU Leuven
Leuven, Belgium
quinten.norga@esat.kuleuven.be

Suparna Kundu
COSIC
KU Leuven
Leuven, Belgium
suparna.kundu@esat.kuleuven.be

Ingrid Verbauwhede
COSIC
KU Leuven
Leuven, Belgium
ingrid.verbauwhede@esat.kuleuven.be

***Abstract*—ML-DSA is a post-quantum lattice-based digital signature algorithm (DSA) that the National Institute of Standards and Technology (NIST) recently standardized as FIPS 204. Remarkably, there are only a handful of published hardware designs and no open-source hardware implementations of complete ML-DSA. In this work, we present an efficient open-source hardware (OSH) design of ML-DSA, based on a Dilithium implementation by Beckwith et al. (FPT 2021). We discuss the required modifications for migrating existing CRYSTALS-Dilithium implementations to match FIPS 204. In addition, we evaluate and compare the performance of our design with the prior art. Through optimized instruction scheduling in the ML-DSA rejection loop, which enables the pre-computation of critical variables, the average signing latency is improved by $16 - 36\%$. Finally, we extensively discuss potential applications and directions of research, further enabled through ML-DSA-OSH.**

***Index Terms*—PQC, ML-DSA, FPGA, Open-Source Hardware**

I. Introduction

Most of the widely deployed public-key cryptographic algorithms are based on RSA [1] and elliptic-curve cryptography (ECC) [2]. However, classical public-key cryptographic schemes are vulnerable against an adversary with large-scale quantum computers due to Shor's algorithm [3]. Therefore, the National Institute of Standards and Technology (NIST) organized the Post-Quantum Cryptography (PQC) standardization procedure in 2016. After several rounds, NIST's first PQC standardization effort officially concluded recently with three Digital Signature Algorithms (DSAs): CRYSTALS-Dilithium (Dilithium v3) [4], Falcon [5], and Sphincs+ [6], and two Key-Encapsulation Mechanisms (KEMs): CRYSTALS-Kyber [7] and HQC [8]. CRYSTALS-Dilithium and CRYSTALS-Kyber are lattice-based schemes and are based on the Module Learning with Errors (MLWE) hard problem. They provide a nice balance between performance, key sizes, and bandwidth requirements. Therefore, CRYSTALS-Dilithium is selected as the primary standard for post-quantum DSA, and CRYSTALS-Kyber is selected as the primary standard for post-quantum KEM. Very recently, NIST published Federal Information Processing Standard (FIPS) specifications for CRYSTALS-Dilithium (FIPS 204) and CRYSTALS-Kyber (FIPS 203) and renamed them ML-DSA [9] and ML-KEM [10], respectively. Although ML-DSA and Dilithium v3 share most of the primitive components, there are some differences between them. This is also the case for ML-KEM and Kyber. However, compared to ML-KEM, significantly less research has been performed on the implementation aspects of ML-DSA.

As the standard has been selected, the industry is encouraged to migrate rapidly towards quantum-resistant cryptography. The real-world applications of digital signature algorithms are enormous. It is used in all applications where the authenticity and integrity of service providers, users, or data are necessary. It is deployed in Transport Layer Security (TLS) to verify the identity of the server, in blockchain to attain integrity of the transaction and non-repudiation, in the financial/ banking sector for security, in the government and public sector to provide trust and maintain integrity, in software libraries, firmware, or software operating systems to secure integrity, in secure emailing for authentication, in health care for providing privacy and integrity, and many more [11]. These applications have different requirements; some have execution time constraints, some have limited hardware resources, and some have limited bandwidth. Although several software implementations of ML-DSA are available and made open-source, only a handful of the Field Programmable Gate Array (FPGA) designs of ML-DSA are published, and almost no FPGA (RTL) implementations of complete ML-DSA are open-source. Open-source FPGA implementations will be beneficial in prototyping or optimizing the implementation of ML-DSA for specific cryptographic applications. Additionally, side-channel attacks and fault-injection attacks are potential threats for an implementation of ML-DSA when it is implemented on a resource-constrained device and if that device is physically accessible by an adversary. An open-source hardware implementation aids in research on side-channel attack analysis and allows for the direct integration countermeasures to prevent them.

A. Related Work

Open-source hardware implementations of Dilithium were presented by Beckwith et al. [12] and Land et al. [13]. The first one proposes a high-performance design, whereas the latter presents a low-area design, at the cost of reduced performance. Additionally, the CHIPS Alliance project has released their PQC IP core (Adam's Bridge), containing an ML-DSA implementation [14] which suffers from a high memory (BRAM + FF) and area footprint. Alternatively, several designs were proposed for which the source code is not made available. A unified, low-area design was presented by Zhao et al. [15]. A lightweight HW implementation supporting only NIST-V was proposed by Gupta et al. [16]. More recently, an implementation balancing performance and area utilization was proposed by Wu et al. [17]. A masked ML-DSA hardware design was proposed in [18], but the source-code is not made publicly available.

B. Contribution

We propose an efficient hardware implementation of the NIST DSA standard: ML-DSA (FIPS 204). First, carefully analyze its algorithms and parameters and show how and where they differ from its Round 3 version (CRYSTALS-Dilithium). Our analysis serves as a guide for anyone implementing/upgrading an existing Dilithium implementation to align with FIPS 204. We present a 'combined' implementation, which supports all ML-DSA operations (`KeyGen`, `SigGen` and `SigVer`) for all parameter sets (NIST II, III & V). The total utilization of our design is 54,942/29,309/29/16 LUTs/FFs/BRAMs/DSPs. Our source code (RTL) is available open-source[1]. We reduce the latency (clock cycles, average) for signature generation (NIST II, III & V) by 16% - 36% compared to prior work through optimized operation scheduling. Finally, we lay out a wide range of directions for future work, including proto-typing and evaluation of (side-channel) countermeasures & attacks and performance optimizations.

II. Preliminaries

This section gives an overview of the three main functions in ML-DSA: key generation, signature generation and signature verification. For a description of all subfunctions, we refer to FIPS 204 [9]. Additionally, we highlight the main differences between the CRYSTALS-Dilithium Round 3 (v3.1) [4] specification and ML-DSA (FIPS 204) [9].

A. Notation and Parameters

q denotes the ML-DSA prime $2^{23} - 2^{13} + 1 = 8380417$. A byte, representing the set of integers $\{0, 1, ..., 255\}$ is represented by $\mathbb{B}$. The ring of integers modulo m whose set of elements is $\{0, 1, ..., m-1\}$ is represented by $\mathbb{Z}_m$. The ring $\prod_{j=0}^{255} \mathbb{Z}_q$ is denoted by T_q and a multiplication of $a, b \in T_q$ is denoted by $a \circ b$. The ring of single-variable polynomials over the ring of integers $\mathbb{Z}$ modulo $X^{256}+1$ is represented by R. The ring of single-variable polynomials over $\mathbb{Z}_m$ modulo $X^{256}+1$ is represented by R_m. All vectors and matrices with elements in R or R_m are denoted by bold lowercase letters (e.g., $\mathbf{v}$) and bold upper case letters (e.g., $\mathbf{A}$), respectively. The amount of elements L in a vector/matrix are indicated in superscript (e.g., $\mathbf{v}^L$). In pseudocode, $s \leftarrow x$ denotes a variable s is assigned the value of x. The sampling of l random bytes is represented by $s \leftarrow \mathbb{B}^l$. Following the notation in FIPS 204, a temporary value that stores the output of a computation $f(x)$ is written as $\ll f(x) \gg$. The symbol $\perp$ indicates the execution of an algorithm failed or there is no output. Additionally, a variable in its NTT form is denoted by a circumflex (e.g., $\hat{\mathbf{A}}$. Computing the infinity norm of an integer is represented by $||\cdot||_\infty$ and a comparison operator inside double square brackets means the expression should be evaluated as a Boolean ($True$ or $False$). The use of symmetric primitive SHAKE256 is represented by the function $\mathtt{H}(str, l)$, with input str and l-byte output.

All three parameter sets of ML-DSA are given in Table I.

TABLE I
ML-DSA parameter sets for NIST security levels II, III and V

Parameters	ML-DSA-44	ML-DSA-65	ML-DSA-87
q - modulus	8380417	8380417	8380417
η - private key range	2	4	2
(k, l) - dimensions of $\mathbf{A}$	$(4, 4)$	$(6, 5)$	$(8, 7)$
d - # dropped bits from $\mathbf{t}$	13	13	13
τ - # of ± 1's in c	39	49	60
λ - collision strength of $\tilde{c}$	128	192	256
γ_1 - coefficient range of $\mathbf{y}$	2^{17}	2^{19}	2^{19}
γ_2 - low-order rounding range	$(q-1)/88$	$(q-1)/32$	$(q-1)/32$
$\beta = \tau \cdot \eta$	78	196	120
ω - max # 1's in $\mathbf{h}$	80	55	75

B. ML-DSA Key Generation

The (internal) key generation algorithm (Alg. 1) generates a public-private keypair (pk, sk) from a 32-byte random seed (ξ). The external algorithm checks the validity of the random seed, before calling the internal algorithm and performing the actual key generation. The seed is first extended using a XOF (SHAKE256) to (ρ, ρ', K), a public random seed, and two private random seeds, respectively. ρ is used to sample the (polynomial) matrix $\hat{\mathbf{A}}$ (in NTT form), ρ' is used to sample polynomials $\mathbf{s_1}$ and $\mathbf{s_2}$, and K is part of the secret key and used during signing. Further, the public value $\mathbf{t}$ is computed as

$$\mathbf{t} = \mathbf{A}\mathbf{s_1} + \mathbf{s_2}$$

which is part of the public key in its compressed form $\mathbf{t_1}$. The d least significant bits, which are dropped here, are included (encoded form) in the secret key as $\mathbf{t_0}$.

C. ML-DSA Signature Generation

The signature generation algorithm (Alg. 2 & 3) generates a signature σ from a secret key sk, message M and optional context string ctx. The message is first formatted to M', before the actual signing procedure commences.

NIST specifies both a hedged (default) and deterministic variant of the signing: either a 32-byte random value rnd is generated and used during signing or it is kept zero,

[1]https://github.com/KULeuven-COSIC/ML-DSA-OSH

Algorithm 1 ML-DSA.KeyGen_internal(ξ)

Generates a public-private key pair from a seed.

Input: Seed $\xi \in \mathbb{B}^{32}$
Output: Public key $pk \in \mathbb{B}^{32+32k(\text{bitlen}(q-1)-d)}$
Output: Private key $sk \in \mathbb{B}^{32+32+64+32((l+k)\text{bitlen}(2\eta)+dk)}$

$(\rho, \rho', K) \in \mathbb{B}^{32} \times \mathbb{B}^{64} \times \mathbb{B}^{32} \leftarrow \mathtt{H}(\xi||\mathtt{IntegerToBytes}(k,1)||\mathtt{IntegerToBytes}(l,1),128)$
$\hat{\mathbf{A}} \leftarrow \mathtt{ExpandA}(\rho)$
$(\mathbf{s}_1, \mathbf{s}_2) \leftarrow \mathtt{ExpandS}(\rho')$
$\mathbf{t} \leftarrow \mathtt{NTT}^{-1}(\hat{\mathbf{A}} \circ \mathtt{NTT}(\mathbf{s}_1)) + \mathbf{s}_2$
$(\mathbf{t}_1, \mathbf{t}_0) \leftarrow \mathtt{Power2Round}(\mathbf{t})$
$pk \leftarrow \mathtt{pkEncode}(\rho, \mathbf{t}_1)$
$tr \leftarrow \mathtt{H}(pk, 64)$
$sk \leftarrow \mathtt{skEncode}(\rho, K, tr, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0)$
return (pk, sk)

respectively [9, Section 3.4]. This fresh randomness can be used during signing, and helps ensure a signer's commitment value $\mathbf{y}$ is not used to sign more than one message and cannot be easily guessed. The lack of randomness in the deterministic variant increases the risk of active side-channel attacks and thus this scenario prefers the hedged variant. Interestingly, a message signed by either the hedged or deterministic signing procedure can be verified using the same verification algorithm.

Algorithm 2 ML-DSA.Sign(sk, M, ctx)

Generates an ML-DSA signature.

Input: Private key $sk \in \mathbb{B}^{32+32+64+32((l+k)\text{bitlen}(2\eta)+dk)}$
Input: Message $M \in \{0,1\}^*$
Input: Context string $ctx \in \mathbb{B}^x$ with $1 \leq x \leq 255$
Output: Signature $\sigma \in \mathbb{B}^{\lambda/4+32l(1+\text{bitlen}(\gamma_1-1))+\omega+k}$

if $|ctx| > 255$ **then**
  return $\perp$
end if
$rnd \leftarrow \mathbb{B}^{32}$ or $\{0\}^{32}$
if $rnd =$NULL **then**
  return $\perp$
end if
$M' \leftarrow \mathtt{BytesToBits}(\mathtt{IntegerToBytes}(0,1)||\mathtt{IntegerToBytes}(|ctx|,1)||ctx)||M$
$\sigma \leftarrow$ ML-DSA.Sign_internal(sk, M', rnd)
return σ

We now describe the signing algorithm in more detail (Alg. 3). Before the (formatted) message M' is signed, the message representative μ is computed by hashing the public key hash tr and the message. Additionally, a private random seed ρ'' is computed using the random (or constant) value rnd, μ and secret K. A valid signature is generated through rejection sampling: a commitment $\mathbf{w}_1$ is computed, a pseudo-random challenge c is derived, and a response $\mathbf{z}$ is computed. Together with a hint $\mathbf{h}$, they are used to compose a signature σ, which is released when it is valid. On average, the signing algorithm will require $4.25, 5.1$ and 3.85 loop iterations for all three variants, respectively.

Algorithm 3 ML-DSA.Sign_internal(sk, M', rnd)

Deterministic algorithm to generate a signature for a formatted message M'.

Input: Private key $sk \in \mathbb{B}^{32+32+64+32((l+k)\text{bitlen}(2\eta)+dk)}$
Input: Formatted message $M' \in \{0,1\}^*$
Input: Random (or dummy) string $rnd \in \mathbb{B}^{32}$
Output: Signature $\sigma \in \mathbb{B}^{\lambda/4+32l(1+\text{bitlen}(\gamma_1-1))+\omega+k}$

$(\rho, K, tr, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0) \leftarrow \mathtt{skDecode}(sk)$
$\hat{\mathbf{s}}_1 \leftarrow \mathtt{NTT}(\mathbf{s}_1)$
$\hat{\mathbf{s}}_2 \leftarrow \mathtt{NTT}(\mathbf{s}_2)$
$\hat{\mathbf{t}}_0 \leftarrow \mathtt{NTT}(\mathbf{t}_0)$
$\hat{\mathbf{A}} \leftarrow \mathtt{ExpandA}(\rho)$
$\mu \leftarrow \mathtt{H}(\mathtt{BytesToBits}(tr)||M', 64)$
$\rho'' \leftarrow \mathtt{H}(K||rnd||\mu, 64)$
$\kappa \leftarrow 0$
$(\mathbf{z}, \mathbf{h}) \leftarrow \perp$
while $(\mathbf{z}, \mathbf{h}) = \perp$ **do**
  $\mathbf{y} \in R_q^l \leftarrow \mathtt{ExpandMask}(\rho'', \kappa)$
  $\mathbf{w} \leftarrow \mathtt{NTT}^{-1}(\hat{\mathbf{A}} \circ \mathtt{NTT}(\mathbf{y}))$
  $\mathbf{w}_1 \leftarrow \mathtt{HighBits}(\mathbf{w})$
  $\tilde{c} \leftarrow \mathtt{H}(\mu||\mathtt{w1Encode}(\mathbf{w}_1), \lambda/4)$
  $c \in R_q \leftarrow \mathtt{SampleInBall}(\tilde{c})$
  $\hat{c} \leftarrow \mathtt{NTT}(c)$
  $\ll c\mathbf{s}_1 \gg \leftarrow \mathtt{NTT}^{-1}(\hat{c} \circ \hat{\mathbf{s}}_1)$
  $\ll c\mathbf{s}_2 \gg \leftarrow \mathtt{NTT}^{-1}(\hat{c} \circ \hat{\mathbf{s}}_2)$
  $\mathbf{z} \leftarrow \mathbf{y} + \ll c\mathbf{s}_1 \gg$
  $\mathbf{r}_0 \leftarrow \mathtt{LowBits}(\mathbf{w} - \ll c\mathbf{s}_2 \gg)$
  if $||\mathbf{z}||_\infty \geq \gamma_1 - \beta$ **or** $||\mathbf{r}_0||_\infty \geq \gamma_2 - \beta$ **then**
    $(\mathbf{z}, \mathbf{h}) \leftarrow \perp$
  else
    $\ll c\mathbf{t}_0 \gg \leftarrow \mathtt{NTT}^{-1}(\hat{c} \circ \hat{\mathbf{t}}_0)$
    $\mathbf{h} \leftarrow \mathtt{MakeHint}(-\ll c\mathbf{t}_0 \gg, \mathbf{w} - \ll c\mathbf{s}_2 \gg + \ll c\mathbf{t}_0 \gg)$
    if $||\ll c\mathbf{t}_0 \gg||_\infty \geq \gamma_2$ **or** $\#1$'s in $\mathbf{h} > \omega$ **then**
      $(\mathbf{z}, \mathbf{h}) \leftarrow \perp$
    end if
  end if
  $\kappa \leftarrow \kappa + l$
end while
$\sigma \leftarrow \mathtt{sigEncode}(\tilde{c}, \mathbf{z} \bmod^{\pm} q, \mathbf{h})$
return σ

Additionally, FIPS 204 specifies a Pre-Hash sign (and verification) variant in case the message M to be signed is large. Instead of signing the message itself, a digest of the message is computed and signed, improving performance.

D. ML-DSA Signature Verification

The signature verification algorithm in FIPS 204 consists of an external and internal function (Alg. 4 & 5). The algorithm checks if the provided signature σ of message M was

generated using the secret key corresponding to pk. Similarly to signing, the message is formatted with a provided context string ctx.

Algorithm 4 ML-DSA.Verify(pk, M, σ, ctx)

Verifies a signature σ for a message M.

Input: Public key $pk \in \mathbb{B}^{32+32k(\text{bitlen}(q-1)-d)}$
Input: Message $M \in \{0,1\}^*$
Input: Signature $\sigma \in \mathbb{B}^{\lambda/4+32l(1+\text{bitlen}(\gamma_1-1))+\omega+k}$
Input: Context string $ctx \in \mathbb{B}^x$ with $1 \leq x \leq 255$
Output: $True$ or $False$

1: **if** $|ctx| > 255$ **then**
2: **return** $\perp$
3: **end if**
4: $M' \leftarrow \texttt{BytesToBits}(\texttt{IntegerToBytes}(0,1)||$
$\texttt{IntegerToBytes}(|ctx|,1)||ctx)||M$
5: **return** ML-DSA.Verify_internal(pk, M', σ)

In the internal verification algorithm, the verifier extracts all components from pk and σ. They attempt to reconstruct the signer's commitment $\mathbf{w}_1$ from the public key and signature, by rounding $\mathbf{w} = \mathbf{Ay}$. Only if the signer's hint $\mathbf{h}$ is valid and the commitment hashes $\tilde{c}$ and $\tilde{c}'$ match, the verification algorithm returns true.

Algorithm 5 ML-DSA.Verify_internal(pk, M', σ)

Internal function to verify a signature σ for a formatted message M'.

Input: Public key $pk \in \mathbb{B}^{32+32k(\text{bitlen}(q-1)-d)}$
Input: Formatted message $M' \in \{0,1\}^*$
Input: Signature $\sigma \in \mathbb{B}^{\lambda/4+32l(1+\text{bitlen}(\gamma_1-1))+\omega+k}$
Output: $True$ or $False$

1: $(\rho, \mathbf{t}_1) \leftarrow \texttt{pkDecode}(pk)$
2: $(\tilde{c}, \mathbf{z}, \mathbf{h}) \leftarrow \texttt{SigDecode}(\sigma)$
3: **if** $\mathbf{h} = \perp$ **then**
4: **return** $False$
5: **end if**
6: $\hat{\mathbf{A}} \leftarrow \texttt{ExpandA}(\rho)$
7: $tr \leftarrow \texttt{H}(pk, 64)$
8: $\mu \leftarrow \texttt{H}(\texttt{BytesToBits}(tr)||M', 64)$
9: $c \in R_q \leftarrow \texttt{SampleInBall}(\tilde{c})$
10: $\mathbf{w}'_{\text{Approx}} \leftarrow \texttt{NTT}^{-1}(\hat{\mathbf{A}} \circ \texttt{NTT}(\mathbf{z}) - \texttt{NTT}(c) \circ \texttt{NTT}(\mathbf{t}_1 \cdot 2^d))$
11: $\mathbf{w}'_1 \leftarrow \texttt{UseHint}(\mathbf{h}, \mathbf{w}'_{\text{Approx}})$
12: $\tilde{c}' \leftarrow \texttt{H}(\mu||\texttt{w1Encode}(\mathbf{w}'_1), \lambda/4)$
13: **return** [[$||\mathbf{z}||_\infty \leq \gamma_1 - \beta$]] **and** [[$\tilde{c} = \tilde{c}'$]]

E. Differences with CRYSTALS-Dilithium

This following list could serve as a starting point when upgrading an implementation of Dilithium to align with the NIST standardized ML-DSA. All changes are explicitly highlighted in the colour red.

1) Domain Separation: To ensure independence between difference instantiations of some functions, domain separation was added to several algorithms in FIPS 204.

a) Seed Expansion (KeyGen): To ensure no misuse when keys for different parameter sets are expanded from the same seed, the seed expansion includes domain separation (old: [4, Fig. 4 (Line 2)], new: [9, Alg. 6 (Line 1)]):

$$(\rho, \rho', K) \in \mathbb{B}^{32} \times \mathbb{B}^{64} \times \mathbb{B}^{32} \leftarrow \texttt{H}(\xi||\texttt{IntegerToBytes}(k,1)||\texttt{IntegerToBytes}(l,1), 128)$$

As a result, the hash function H (SHAKE256) has to absorb a string of 34 bytes, instead of 32 bytes previously.

b) Context String ctx & Formatted Message M' (Sign): NIST specifies the signing algorithm can include a context string, which is empty by default but may be used in some applications as a domain separator in the used hash functions [9, Alg. 2 (Line 1-3)]. As a result, the internal signing procedure uses a formatted message M' instead of the message M directly [9, Alg. 2 (Line 10)], as was the case in Dilithium 3.1.

$$M' \leftarrow \texttt{IntegerToBytes}(0,1)||\texttt{IntegerToBytes}(|ctx|,1)||ctx)||M$$

2) Parameters and Variables: The length (and naming) of several parameters were changed between the release of Dilithium 3.1 and ML-DSA. Below, we list those that impact the functionality of the implementation.

a) Public Key Hash tr (KeyGen, Sign & Verify): To ensure the properties beyond strong unforgeability [19], a 64-byte hash of the public key (tr) [9, Alg. 6 (Line 9)] is added to the secret key sk (instead of 32-byte [4, Fig. 4 (Line 9)]). This message representative is used during signing, instead of directly signing the message.

$$tr \leftarrow \texttt{H}(pk, 64)$$

b) Commitment hash $\tilde{c}$ (Sign & Verify): The message representative μ and commitment $\mathbf{w}_1$ are concatenated and hashed to obtain the commitment hash. For ML-DSA-65 and ML-DSA-87, the length of $\tilde{c}$ is increased to 384 and 512 bits, respectively [9, Alg.7 (Line 15)]. In the Dilithium specification, this value was fixed for all parameter sets at 256 bits [4, Fig. 4 (Line 17)].

$$\tilde{c} \leftarrow \texttt{H}(\mu, \texttt{w1Encode}(\mathbf{w}_1), \lambda/4)$$

3) Hedged and Deterministic Signing: FIPS 204 specifies a hedged and deterministic variant of the signing procedure [9, Alg. 2 (Line 5)] (see Section II-C). The hedged variant requires the generation of a 32-byte random string during signing:

$$rnd \leftarrow \mathbb{B}^{32}$$

Alternatively, it is set to a constant (zero) string for the deterministic variant:

$$rnd \leftarrow \{0\}^{32}$$

a) Private Random Seed ρ'' (Sign): This private seed is used in the generation of $\mathbf{y}$, which is used by the signer to generate the commitment $\mathbf{w}_1$ and must not be used to sign more than one message. In order to ensure it cannot be guessed easy by an adversary (exploiting side-channels), ρ'' is generated from the secret K, the message representation μ and a random string rnd. This value is either randomly generated or zero (constant), for hedged and deterministic signing, respectively.

$$\rho'' \leftarrow \texttt{H}(K \parallel rnd \parallel \mu, 64)$$

III. Proposed Architecture and Optimizations

The following section covers the hardware architecture and design of all main sub-modules of our implementation. They are combined to construct a unified architecture that supports key generation, signature generation, and signature verification at all three NIST levels.

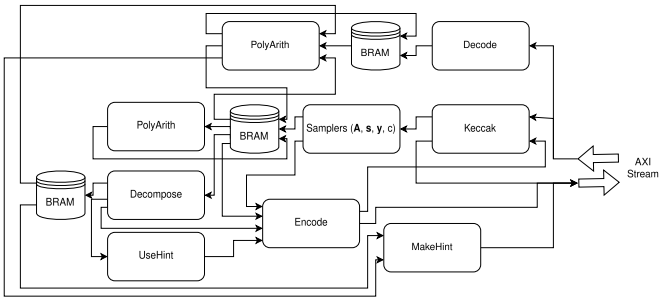

Fig. 1. High-level architecture for ML-DSA, based on [12]. The main arithmetic units and storage elements (BRAM) are shown.

A. High-level Architecture

Our implementation is modified from the high-speed CRYSTALS-Dilithium design proposed by Beckwith et al. [12]. We integrate all modifications presented in Section II-E, as well as increase the robustness of the design to support the signing of a wider range of message lengths. A high-level overview of the architecture is shown in Figure 1.

The authors optimize performance by: (1) optimizing polynomial arithmetic operators and (2) splitting the rejection sampling loop in the signature generation operation into a two-stage pipeline. To store intermediate results, the authors propose to use (true dual port) BRAM units, which can access four polynomial samples per cycle ($4 \times 24 = 96$ bits), ensuring all arithmetic units are not stalled.

B. Arithmetic Operators

The arithmetic operator modules are used to compute all polynomial arithmetic: addition, subtraction, and Number Theoretic Transform (NTT)-based multiplication. The modular multiplier uses Barrett reduction, requiring only shifts and additions in hardware. The butterfly units support both Cooley-Tukey and Gentlemen-Sande operations for NTT and INTT operations. The authors rely on a 2×2 butterfly arrangement, which can process four coefficients in parallel, per clock cycle. Such a configuration was proposed in [20] and provides a trade-off between memory access stalls by processing two NTT layers at once, on the one hand. On the other hand not instantiating 16 butterfly units in a 4×4 configuration would come with a large penalty in area overhead. In the proposed design, the total latency is reduced by minimizing the number of write-back conflicts and appropriately ordering the coefficients. The butterfly units are implemented on the FPGA using DSP units to ensure a sufficiently high operating clock frequency. We refer to the original work [12] for an in-depth description of the coefficient ordering, using a custom address resolver, and memory architecture.

C. Polynomial Sampler

Being a costly operation in lattice-based schemes, the authors in [12] propose to instantiate three Keccak (SHAKE256) cores for the polynomial sampling and hashing. Two units are dedicated to sampling the public matrix $\mathbf{A}$, and the third one is for all other sampling and hashing operations. Sufficient memory (BRAM) is instantiated to ensure the public matrix can be fully stored, minimizing stalling during the signing procedure. We modify the design to follow ML-DSA specs, and include domain separators where needed. Additionally, we ensure the random string rnd is used during the generation of the private random seed ρ'' during signing.

D. Operation Scheduling

ML-DSA-OSH follows a centralized processor architecture, in which most of the operation scheduling is controlled through a single state machine. The operations are scheduled such that the arithmetic units are maximally occupied (minimal stalling) and consume the data (inputs) as soon as possible.

1) Key Generation: The generation of $\mathbf{t}$ requires the sampling of matrix $\mathbf{A}$, $\mathbf{s_1}$ & $\mathbf{s_2}$, as well as an `NTT`, multiplication, `INTT` and addition (Section II-B). The sampling is immediate, followed by the polynomial arithmetic and subsequently the encoding and hash operation to generate tr.

2) Signature Generation: The authors of [12] propose to split the signature generation into three stages: a pre-computation phase (stage 1) and two rejection loop stages (stage 2-3). The pre-computation phase decodes the secret key components and transforms them to the NTT domain, the first rejection loop stage generates the commitment $\mathbf{w}$, and the second computes the response $\mathbf{z}$ and hint $\mathbf{h}$. The pre-computation stage runs in parallel with the first rejection loop stage, after which that result is used in the final rejection loop stage. Simultaneously, the first rejection loop stage starts computing a new commitment early, in case not all conditions are met in the final stage (reject). This corresponds to the while loop in Section II-C skipping to the next iteration. We optimize the total latency of the signature generation by ensuring the the polynomial multiplication between public matrix $\mathbf{A}$ and $\mathbf{y}$ (stage 2) starts as soon as both components have been generated in the first stage. Previously, for the medium and high-security version of Dilithium, the architecture would unnecessarily stall when the `NTT` computation of $\mathbf{y}$ exceeded the sampling of $\mathbf{A}$.

IV. Performance Evaluation

We now present the performance results of our ML-DSA hardware (FPGA) implementation which support key generation, signature generation and signature verification for all three NIST security categories. We verify the functional correctness of our implementation using the NIST ACVP testvectors and deterministic signing variant (see Section II-C). This allows for a fair comparison with prior art and only minor modifications are required to support the hedged variant.

A. Area Results

All results are generated using the Xilinx Vivado 2021.1 tool, targeting the ZCU102 Ultrascale+ platform. We present the results for our *combined* design, which supports all parameter sets at runtime. As shown in Table II, the Keccak cores and NTT modules are the largest components (50% of total utilization). As they contribute most to total latency, the operations can be sped up by parallelizing them and investing in hardware resources. Our combined, high-speed implementation requires 54,942 LUTs and 29,309 FFs. To store intermediate results, 29 BRAMs (36 Kb) are used, and the arithmetic units use 16 DSP blocks.

TABLE II
Resource utilization of ML-DSA and its sub-modules on FPGA.

Sub-module	LUTs	FFs	DSPs	BRAMs
Samplers $\{\mathbf{A}, \mathbf{s}, \mathbf{y}, c\}$	8,241	3,230	0	0
MakeHint & UseHint	8,837	3,772	0	0
PolyArith (NTT)	8,794	5,889	16	0
Keccak	18,678	13,394	0	0
(D)e(n)code & Decompose	6,889	1,368	0	0
Data Memory	2,317	0	0	29
Control & Other	1,186	1,656	0	0
Total	54,942	29,309	16	29

B. Performance Comparison

Table III provides the area and performance results of our designs and prior work. We note that different works target different optimization goals (low-area, high-performance, ...) and use different

TABLE III
OVERVIEW AND COMPARISON OF PQC DSA HARDWARE IMPLEMENTATIONS.

Scheme	Works	Platform	Latency [cc & μs] KeyGen	SigGen	SigVer	Area [LUT/FF/DSP/BRAM] (or mm^2 for ASIC)
Dilithium-II (v3.1)	[13]	A7 @ 163 MHz	18,761 115	76,613 470	19,687 121	27,433/10,681/45/15
	[17]	A7 @ 114.6 MHz	5,251 46	33,503 293	4,936 43	20,621/8,104/16/8
Dilithium-III (v3.1)	[13]	A7 @ 145 MHz	33,102 228	123,218 850	32,050 221	30,900/11,372/45/21
	[17]	A7 @ 109 MHz	7,871 72	53,577 491	7,610 70	23,378/9,079/8/20
Dilithium-V (v3.1)	[13]	A7 @ 140 MHz	50,982 363	145,192 1,042	52,712 377	44,653/13,814/45/31
	[17]	A7 @ 114.3 MHz	12,483 109	72,179 632	11,825 103	28,791/11,338/24/8
	[16]	US+ @ 391 MHz	63,200 161	113,900 291	67,900 173	13,975/6,845/35/4
ML-DSA-87	[14]	5nm @ 600 MHz	15,600 26	106,400 177	18,800 31	0.0366
Dilithium (v3.1)	[12]	US+ @ 256 MHz	4,875/8,291/14,037 19/32/55	29,876/49,437/55,070 117/193/215	6,582/9,724/13,642 26/38/53	53,907/28,435/16/29
	[15]	A7 @ 96.9 MHz	4,172/5,851/8,765 43/60/90	28,091/44,706/48,996 290/461/506	4,422/6,181/9,039 46/64/93	29,998/10,366/11/10
ML-DSA	**Ours**	US+ @ 230 MHz	4,872/8,291/14,033 21/36/61	24,283/32,800/37,592 106/143/160	6,637/9,730/14,642 29/42/64	54,942/29,309/16/29

target platforms when reporting performance. As a result, a direct and fair comparison between different hardware implementations is not always possible.

We list the performance of designs supporting the different NIST security levels at compile time. Land et al. [13] propose designs for all NIST security levels, which our design outperforms for all operations with only minor area overhead, supporting all security levels at runtime. The lightweight design by Gupta et al. [16] only supports the highest security version (V) at low area cost. Our design has lower latency for key generation, signature generation, and signature verification. The Adam's Bridge IP core [14], which supports the highest ML-DSA security variant, is a high-performance design for ASIC and has a high area utilization and worse latency for all operations compared to ours. However, this is due to high levels of pipelining, ensuring the design can run at upwards of 600 MHz when synthesized in $5nm$ technology. As a result, their execution time is significantly lower for all operations.

Additionally, we compare all *combined* designs, which support all parameter sets at runtime. As expected, the modifications presented in Section II-E results in a minor ($\pm 1\%$) increase in area overhead, compared to the Dilithium (v3.1) version [12]. Compared to the low-area design by Zhao et al. [15], our designs require $1.83/2.82/1.45/2.9\times$ more LUT/FF/DSP/BRAM. We also present the timing results (latency) of all three signature algorithm routines, for security levels II, III, and V. All our results are obtained when setting the clock constraint to 230 MHz, whereas [12] reports a maximal clock frequency of 256 MHz. For medium security parameters, our design requires 36.05/142.61/63.66 μs for key generation, signature generation (avg.), and signature verification, respectively. Interestingly, our modifications and improvements result in a reduction of 16% - 36% in (average) clock cycles for the signature generation (NIST II, III and V) operation. A contributing factor is ensuring the multiplication $\hat{\mathbf{A}} \circ \texttt{NTT}(\mathbf{y})$ starts as soon as both multiplicands are computed.

V. CONCLUSION AND FUTURE WORK

In this work, we presented an efficient, open-source hardware implementation (ML-DSA-OSH) of the new NIST PQC standard ML-DSA. We achieved this through a thorough analysis of FIPS 204 and by comparing it to the Round 3 specification of CRYSTALS-Dilithium. Our design is highly flexible, supporting all three security levels and digital signature routines. Our improved signature generation instruction scheduling results in signature generation latency of 106/143/160 μs for NIST II, III and V variants.

ML-DSA-OSH, an open-source (RTL) hardware implementation of NIST standardized ML-DSA, directly enables different avenues of future research. While optimized designs of (stand-alone) sub-modules have been proposed in literature, their implementations are rarely open-sourced. This limited access for the wider community reduces the opportunity for follow-up work and hinders the exploration of optimal ML-DSA HW architectures. Our implementation allows for the direct integration (and cost evaluation) of optimized sub-modules, such as: NTT, Sampler and Lightweight Hash/PRNG. Additionally, different high-level architectures can be explored, starting from ML-DSA-OSH, which could suit different use-cases for real-world deployment. This includes extremely high-throughput designs (streaming architectures) or implementations targeting resource-constrained environments/platforms (low-power, low-area).

In addition, many side-channel and fault-injection attacks on Dilithium have been proposed on the Cortex-M4 implementation due to the lack of open-source hardware implementations [21]–[23]. This work enables extending this work to the hardware domain and real-world implementations.

Finally, shuffling and masking are well-known countermeasures for protecting against power analysis attacks, whereas duplication and repetition can be used to prevent the exploitation of fault attacks. A necessary (next) step is the hardening of *hardware* implementations. In the context of Dilithium & ML-DSA, (optimized) masked gadgets have been proposed by Coron et al. [24], [25] and prototyped through software implementations. Compared to ML-KEM/Kyber, for which several optimized hardware countermeasures have been proposed [26], [27], this is not the case for ML-DSA. Similarly, several fault attack countermeasures have been proposed and implemented on Cortex-M4 [28], [29]. An open-question is how well these protect against fault attacks when implemented hardware and how costly their overhead is on these platforms.

ACKNOWLEDGMENT

This work was partially supported by Horizon 2020 ERC Advanced Grant (101020005 Belfort), Cybersecurity Research Program Flanders (CRPF) with reference number VOEWICS02, Belgian-QCI (3E230370) (see beqci.eu) and Intel Corporation.

REFERENCES

[1] R. L. Rivest, A. Shamir, and L. M. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Commun. ACM*, vol. 21, no. 2, pp. 120–126, 1978.

[2] V. S. Miller, "Use of elliptic curves in cryptography," in *Advances in Cryptology — CRYPTO '85 Proceedings*, H. C. Williams, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1986, pp. 417–426.

[3] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *35th Annual Symposium on Foundations of Computer Science, Santa Fe, New Mexico, USA, 20-22 November 1994*. IEEE Computer Society, 1994, pp. 124–134. [Online]. Available: https://doi.org/10.1109/SFCS.1994.365700

[4] S. Bai, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehle, "CRYSTALS-Dilithium: Algorithm Specifications and Supporting Documentation (Version 3.1)," Online. Accessed 10th February, 2025, 2021. [Online]. Available: https://pq-crystals.org/dilithium/data/dilithium-specification-round3-20210208.pdf

[5] P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Prest, T. Ricosset, G. Seiler, W. Whyte, and Z. Zhang, "Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU," 2018, https://falcon-sign.info/.

[6] J.-P. Aumasson, D. J. Bernstein, W. Beullens, C. Dobraunig, M. Eichlseder, S. Fluhrer, S.-L. Gazdag, A. Hülsing, P. Kampanakis, S. Kölbl, T. Lange, M. M. Lauridsen, F. Mendel, R. Niederhagen, C. Rechberger, J. Rijneveld, P. Schwabe, and B. Westerbaan, "SPHINCS+ Submission to the NIST post-quantum project, v.3.1," 2018, https://sphincs.org/data/sphincs+-r3.1-specification.pdf.

[7] R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehle, "CRYSTALS-Kyber Algorithm Specifications And Supporting Documentation (version 3.02)," 2021, https://pq-crystals.org/kyber/data/kyber-specification-round3-20210804.pdf.

[8] C. A. Melchor, N. Aragon, S. Bettaieb, and O. Blazy, "Hamming quasi-cyclic (HQC)," 2018, https://pqc-hqc.org/doc/hqc-specification_2023-04-30.pdf.

[9] NIST, "FIPS 204 Module-Lattice-Based Digital Signature Standard," Online. Accessed 10th February, 2025, 2024. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf

[10] ——, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," Online. Accessed 10th April, 2025, 2024. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf

[11] T. G. Tan, P. Szalachowski, and J. Zhou, "Challenges of post-quantum digital signing in real-world applications: a survey," *Int. J. Inf. Sec.*, vol. 21, no. 4, pp. 937–952, 2022. [Online]. Available: https://doi.org/10.1007/s10207-022-00587-6

[12] L. Beckwith, D. T. Nguyen, and K. Gaj, "High-performance hardware implementation of crystals-dilithium," in *2021 International Conference on Field-Programmable Technology (ICFPT)*, 2021, pp. 1–10.

[13] G. Land, P. Sasdrich, and T. Güneysu, "A hard crystal - implementing dilithium on reconfigurable hardware," in *Smart Card Research and Advanced Applications*, V. Grosso and T. Pöppelmann, Eds. Cham: Springer International Publishing, 2022, pp. 210–230.

[14] CHIPS Alliance, "Adam's bridge," Online. Accessed 18th February, 2025. [Online]. Available: https://github.com/chipsalliance/adams-bridge

[15] C. Zhao, N. Zhang, H. Wang, B. Yang, W. Zhu, Z. Li, M. Zhu, S. Yin, S. Wei, and L. Liu, "A compact and high-performance hardware architecture for crystals-dilithium," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2022, no. 1, p. 270–295, Nov. 2021. [Online]. Available: https://tches.iacr.org/index.php/TCHES/article/view/9297

[16] N. Gupta, A. Jati, A. Chattopadhyay, and G. Jha, "Lightweight hardware accelerator for post-quantum digital signature crystals-dilithium," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 70, no. 8, pp. 3234–3243, 2023.

[17] Z. Wu, R. Chen, Y. Wang, Q. Wang, and W. Peng, "An efficient hardware implementation of crystal-dilithium on fpga," in *Information Security and Privacy*, T. Zhu and Y. Li, Eds. Singapore: Springer Nature Singapore, 2024, pp. 64–83.

[18] K. Raj, P. Ravi, T. K. Chia, and A. Chattopadhyay, "Improved ML-DSA hardware implementation with first order masking countermeasure," Cryptology ePrint Archive, Paper 2024/1817, 2024. [Online]. Available: https://eprint.iacr.org/2024/1817

[19] C. Cremers, S. Düzlü, R. Fiedler, M. Fischlin, and C. Janson, "Buffing signature schemes beyond unforgeability and the case of post-quantum signatures," in *2021 IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 1696–1714.

[20] D. T. Nguyen, V. B. Dang, and K. Gaj, "A high-level synthesis approach to the software/hardware codesign of ntt-based post-quantum cryptography algorithms," in *2019 International Conference on Field-Programmable Technology (ICFPT)*, 2019, pp. 371–374.

[21] Z. Chen, E. Karabulut, A. Aysu, Y. Ma, and J. Jing, "An efficient non-profiled side-channel attack on the crystals-dilithium post-quantum signature," in *39th IEEE International Conference on Computer Design, ICCD 2021, Storrs, CT, USA, October 24-27, 2021*. IEEE, 2021, pp. 583–590. [Online]. Available: https://doi.org/10.1109/ICCD53106.2021.00094

[22] Z. Qiao, Y. Liu, Y. Zhou, J. Ming, C. Jin, and H. Li, "Practical public template attack attacks on crystals-dilithium with randomness leakages," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 1–14, 2023. [Online]. Available: https://doi.org/10.1109/TIFS.2022.3215913

[23] E. Krahmer, P. Pessl, G. Land, and T. Güneysu, "Correction fault attacks on randomized crystals-dilithium," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2024, no. 3, pp. 174–199, 2024. [Online]. Available: https://doi.org/10.46586/tches.v2024.i3.174-199

[24] J.-S. Coron, F. Gérard, M. Trannoy, and R. Zeitoun, "Improved gadgets for the high-order masking of dilithium," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2023, no. 4, p. 110–145, Aug. 2023. [Online]. Available: https://tches.iacr.org/index.php/TCHES/article/view/11160

[25] J.-S. Coron, F. Gérard, T. Lepoint, M. Trannoy, and R. Zeitoun, "Improved high-order masked generation of masking vector and rejection sampling in dilithium," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2024, no. 4, p. 335–354, Sep. 2024. [Online]. Available: https://tches.iacr.org/index.php/TCHES/article/view/11795

[26] Q. Norga, J.-P. D'Anvers, S. Kundu, and I. Verbauwhede, "X2X: Low-randomness and high-throughput A2B and B2A conversions for $d+1$ shares in hardware," Cryptology ePrint Archive, Paper 2024/114, 2024. [Online]. Available: https://eprint.iacr.org/2024/114

[27] R. C. Rodriguez, E. Valea, F. Bruguier, and P. Benoit, "Hardware implementation and security analysis of local-masked NTT for CRYSTALS-kyber," Cryptology ePrint Archive, Paper 2024/1194, 2024. [Online]. Available: https://eprint.iacr.org/2024/1194

[28] S. Bauer, F. D. Santis, K. Koleci, and A. Aghaie, "A fault-resistant NTT by polynomial evaluation and interpolation," Cryptology ePrint Archive, Paper 2024/788, 2024. [Online]. Available: https://eprint.iacr.org/2024/788

[29] M. Duparc and M. Taha, "Improved NTT and CRT-based RNR blinding for side-channel and fault resistant kyber," Cryptology ePrint Archive, Paper 2025/181, 2025. [Online]. Available: https://eprint.iacr.org/2025/181